Report

Project URL: https://github.com/priyanov101999/Grepr-Take-Home

The setup and workflow is present in README.md file

Description

This service exposes a RESTful API that allows authenticated users to execute long-running SQL SELECT queries and retrieve results asynchronously.

Queries may take a long time and return large result sets that do not fit in memory, so execution is handled in the background and results are streamed to clients.

The system includes multiple safeguards to ensure it remains stable under intentional or unintentional abuse.

Design:

- Users submit SQL queries and receive a query ID
- Queries execute asynchronously using worker threads
- Clients poll for status and fetch results when ready
- Results are streamed as NDJSON
- Only SELECT queries are supported

Safety & Abuse Protection Scenarios Handled:

SQL Guard

- The SQL queries are run asynchronously
- The user can query only “SELECT” . Forbidden keywords are insert, update, delete, drop, alter,create, copy, grant, revoke
- Max SQL length enforced
- Multiple statements together cant be executed so no semi colon
- Timeout for query execution is done

Large Result Sets Streaming

- The output is not returned entirely, instead its streams to prevent OOM
- Results returned as NDJSON (row-by-row streaming)

Authentication and Isocation

- All the queries require authentication
- The scope of each query is for each individual user
- The user can only see their query,so cross user access is not allowed

Rate Limiting and Concurrency Limit

- The users are rate limited per minute
- There is a limit for how many queries the user can have are pending and running
- There is a limit for globally running queries in the worker
- There is a limit for maximum length for sql given by the user in query/submit endpoint
- The queue is bounding to prevent overwhelming the server for backpressure

Execution Safety

- There is a limit for the no of rows and the result size returned after the user query, if its exceeds it leads to error
- JDBC streaming is handled
- Query cancellation signals the database to stop execution but does not guarantee immediate termination.
- Structured error responses for user readability

Reliability

- Every query's state is saved in the DB
- Idempotent query submission supported so that same request does not create 2 queries
- Once the query is executed successfully, the results are saved and available

Configuration

Configurations through config.yml:

- DB connection
- Worker count
- Queue size
- Rate limit per minute
- Per-user & global concurrency caps
- Statement timeout
- Fetch size
- Max rows / bytes
- Results directory

Future Improvements

1. Currently I am using string checks, in the future I would use SQL parser for safer and more accurate query validation.
2. Im storing all the results in results directory, in the future would store it in s3 buckets for scalability
3. Currently for authentication, using a mock user id. In the future I would using JWT auth with refresh tokens for real authentication and authorization for RBAC
4. Im using Logs to trace errors now, But adding metrics in the future would improve observability
5. I would prefer to add DB-level read-only enforcement

Results after ./test.sh is executed

```
priyadarshni@priyas-MBP grepr-be % ./test.sh

==================================================
A) Basic health check
==================================================
PASS: GET /ping returns 200 (http 200)
PASS: Ping response contains ok

==================================================
B) Authentication enforcement
==================================================
PASS: Submitting without auth is rejected (http 401)
PASS: Reading query without auth is rejected (http 401)

==================================================
C) Request validation (400 errors)
==================================================
PASS: Missing sql field (http 400)
PASS: Blank sql rejected (http 400)
PASS: Malformed JSON rejected (http 400)

==================================================
D) SQL guard enforcement
==================================================
PASS: Non-select statement rejected (http 400)
PASS: Multiple statements rejected (http 400)
PASS: Forbidden keyword detected (http 400)

==================================================
E) Happy path: submit -> poll -> fetch results
==================================================
{"id":"q_30215084d1e64a929432ef891a40680d","status":"PENDING","createdAt":1766006192.440798000,"startedAt":null,"endedAt":null,"error":null,"rowsWritten":null,"bytesWritten":nu
ll}
PASS: Query submitted (id=q_30215084d1e64a929432ef891a40680d)
Polling until completion...
  status=RUNNING
```

```
{"id":"q_30215004d1e64a929432ef891a40680d","status":"SUCCEEDED","createdAt":1766006192.440798000,"startedAt":1766006192.452933000,"endedAt":1766006192.777225000,"error":null,"r
owsWritten":4,"bytesWritten":134}
PASS: Results available after success (http 200)
PASS: Results stream is valid NDJSON

==================================================
F) Results requested before completion
==================================================
PASS: Results unavailable while query is running (http 409)

==================================================
G) Idempotency handling
==================================================
PASS: Same id returned for idempotent request

==================================================
H) Cross-user isolation
==================================================
PASS: User1 cannot access User2 query (http 404)

==================================================
I) Cancellation flow
==================================================
PASS: Cancel request accepted (http 200)
  cancel_status=CANCELLED
PASS: Query transitioned to CANCELLED

==================================================
J) Unknown query id
==================================================
PASS: Unknown id returns 404 (http 404)

==================================================
K) Rate limiting
==================================================
  request=1 http=200
```

```
==================================================
K) Rate limiting
==================================================
  request=1 http=200
  request=2 http=429
  request=3 http=200
  request=4 http=429
  request=5 http=200
  request=6 http=429
  request=7 http=200
  request=8 http=429
  request=9 http=200
  request=10 http=429
  request=11 http=200
  request=12 http=429
  request=13 http=200
  request=14 http=429
  request=15 http=200
  request=16 http=429
  request=17 http=200
  request=18 http=429
  request=19 http=200
  request=20 http=429
  request=21 http=200
  request=22 http=429
  request=23 http=200
  request=24 http=429
  request=25 http=200
  request=26 http=429
  request=27 http=429
  request=28 http=429
  request=29 http=429
  request=30 http=429
  request=31 http=429
  request=32 http=429
  request=33 http=429
```

```
  request=47 http=429
  request=48 http=429
  request=49 http=429
  request=50 http=429
  request=51 http=429
  request=52 http=429
  request=53 http=429
  request=54 http=429
  request=55 http=429
  request=56 http=429
  request=57 http=429
  request=58 http=429
  request=59 http=429
  request=60 http=429
PASS: Rate limiting observed

All major integration scenarios executed successfully.
priyadarshni@privas-MBP greno-be %
```